

Transfer Learning & Batch Normalization

Neural Networks & Computer Vision Project
Presentation

Presented by: Tanush Shoor (24BCS10265)

Section: B | Group: 9 | Instructor:

Priyansh Saxena



Project Overview

An investigation into efficient training techniques for limited data and deep neural architectures.



Research Objectives



Transfer Learning

Compare ResNet-18 vs. CNN from scratch on a small dataset (1000 images) to overcome low accuracy.



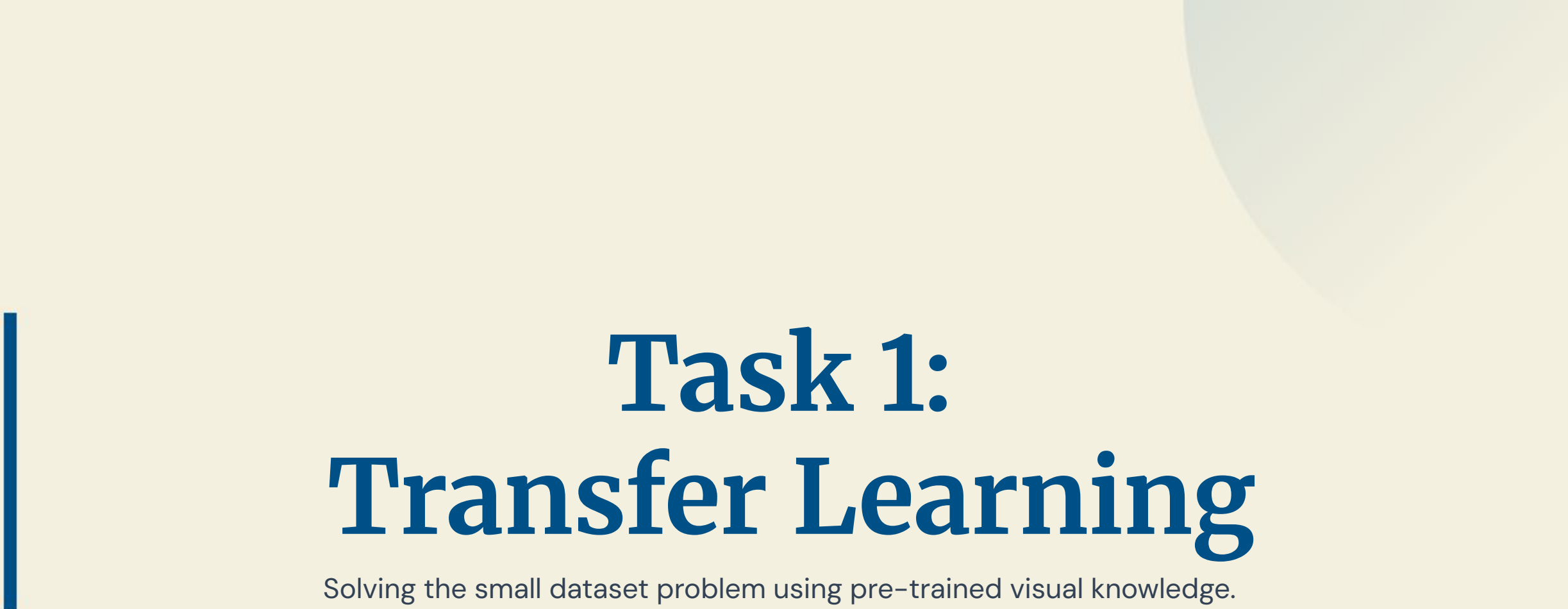
Batch Normalization

Evaluate stability and convergence speed in deep 6+ layer networks using the Fashion-MNIST dataset.



Practical Analysis

Visualize training loss, final classification accuracy, and model stability for VIVA defense.



Task 1: Transfer Learning

Solving the small dataset problem using pre-trained visual knowledge.

Dataset: Cats vs Dogs Mini

-  **Total Images:** 1000 (500 Cats / 500 Dogs).
-  **Split:** 80% Training, 20% Validation.
-  **Preprocessing:** Resized to 224x224 pixels with normalization.
-  **Challenge:** Small data size usually causes CNNs to struggle with feature learning.



Comparative Methodologies



CNN from Scratch

Randomly initialized weights.
Requires the network to learn every visual feature from a small pool of data.



Frozen ResNet-18

Uses ImageNet features.
Backbone layers are locked;
only the final classifier is trained on new data.



Fine-Tuning

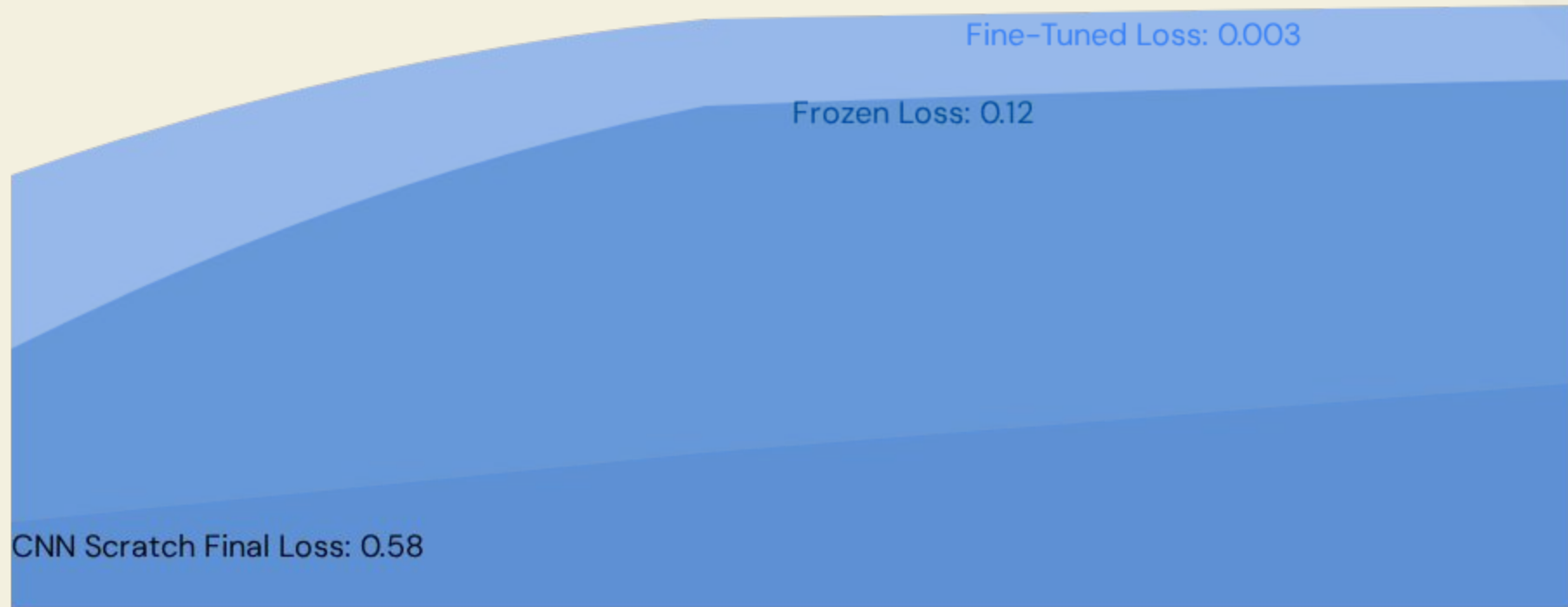
ResNet-18 initialized with pre-trained weights, but the last conv block is updated to adapt to specific features.

Accuracy Results: Task 1

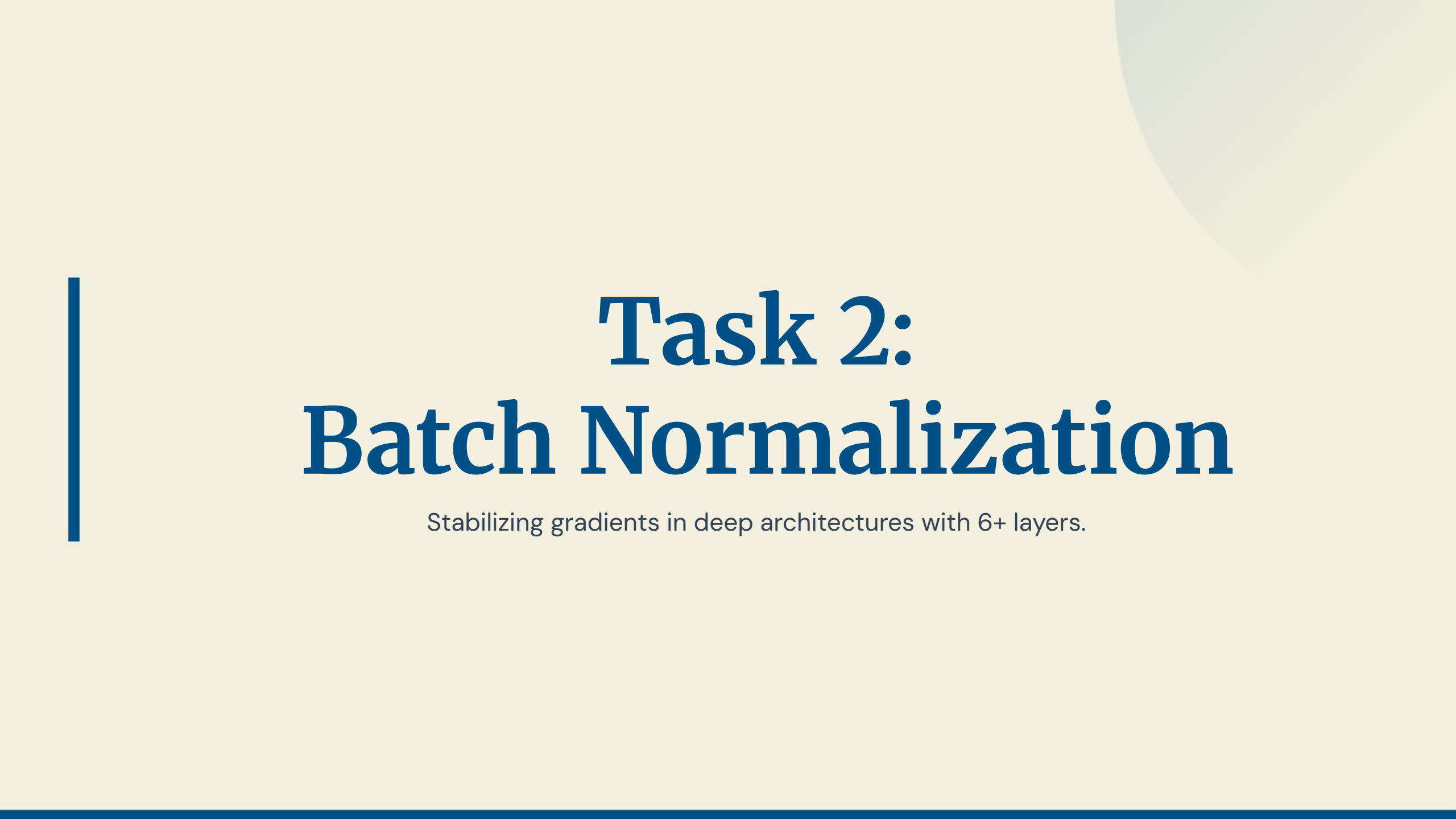


Conclusion: Transfer Learning provides a massive ~38% boost in accuracy over training from scratch.

Loss Convergence Trends



The **Fine-Tuned ResNet** achieves near-zero loss, indicating superior feature extraction.



Task 2: Batch Normalization

Stabilizing gradients in deep architectures with 6+ layers.

Dataset: Fashion-MNIST



Content: 10 Clothing classes (Coat, Bag, Sneaker etc).



Volume: 60,000 Training / 10,000 Test images.



Resolution: 28x28 grayscale pixels.



Goal: Test if normalization helps a 6-layer deep network converge faster.



Pullover (2)



Trouser (1)



Bag (8)



Coat (4)



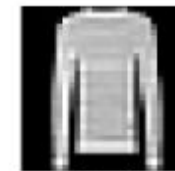
Trouser (1)



Ankle boot (9)



Pullover (2)



Pullover (2)

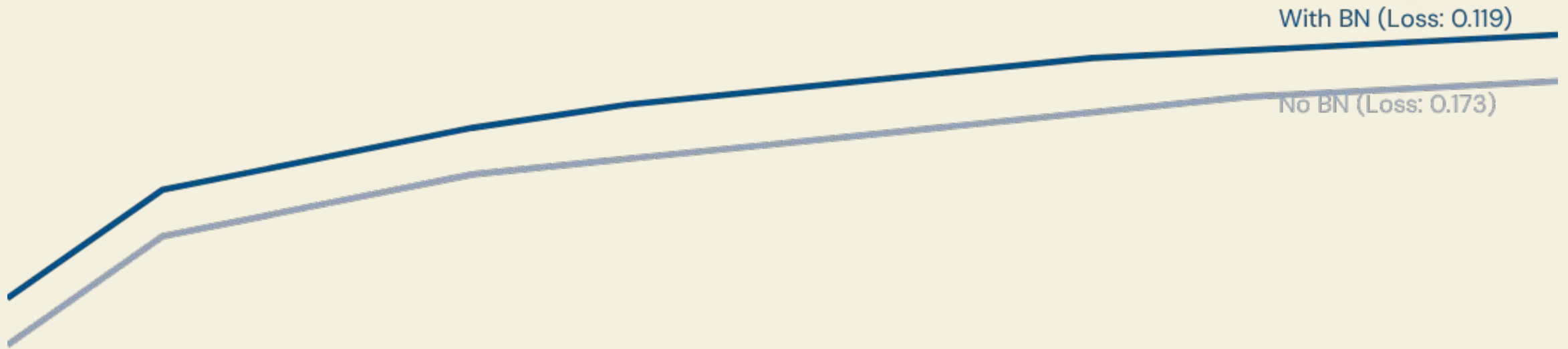


T-shirt/top (0)

Deep Network Architecture (6+ Layers)

- ➡ **Input Layer:** 784 neurons (28x28 flattened).
- ➡ **Hidden Layers (x5):** Sequential fully-connected layers (512 → 256 → 128 → 64 → 32).
- 🛠️ **Batch Normalization:** Inserted after each hidden layer to re-center activations.
- ➡ **Activation:** ReLU used across all stages.
- 🏁 **Output Layer:** 10 neurons with Softmax for classification.

Convergence Comparison (20 Epochs)



The Batch Normalization curve is **consistently lower and smoother**, preventing activation drifts.

Final Impact of Batch Norm

89.57%

Final Test Accuracy

Beyond just Accuracy

While final accuracy improved slightly (up from 88.75%), the real benefit was in **stability**. Training was faster, and the network was less sensitive to weight initialization, allowing for higher learning rates without divergence.

Project Summary: Key Findings

Metric	Project 1: Transfer Learning	Project 2: Batch Norm
Primary Goal	Accuracy on small datasets	Training stability & speed
Dataset	Cats vs Dogs (Mini)	Fashion-MNIST
Best Performer	Fine-Tuned ResNet18 (99.5%)	BatchNorm Net (89.57%)
Takeaway	Reusing knowledge is better than learning from scratch	Normalization prevents distribution shifts in deep nets

Q&A

Thank you for your time!

Neural Networks Course Project
Tanush Shoor | INDIVIDUAL Project | 2024



https://storage.googleapis.com/tfds-data/visualization/fig/cats_vs_dogs-4.0.1.png

Source: www.tensorflow.org

Image Sources



https://storage.googleapis.com/tfds-data/visualization/fig/fashion_mnist-3.0.1.png

Source: www.tensorflow.org

